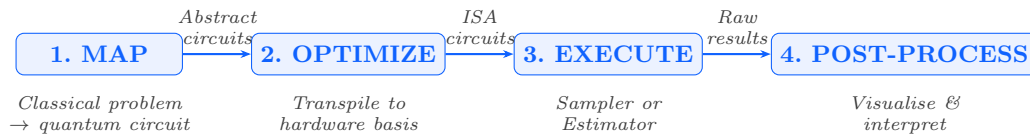


Week 3: Quantum Computing Workflow

The Qiskit Pattern: Map → Optimize → Execute → Post-process

quantum.cloud.ibm.com/docs — Development Workflow (Intro to Patterns)



Step 1 — Map

Translate a **classical domain problem** into quantum circuits and/or operators.

- Choose algorithm structure (e.g. variational, phase estimation)
- Build `QuantumCircuit` objects encoding the problem
- Specify observables as `SparsePauliOp` (for Estimator)
- Decide output type: *bitstrings* (Sampler) or *expectation values* (Estimator)

Example (Bell state):

$$|00\rangle \xrightarrow{H \otimes I} \frac{|0\rangle + |1\rangle}{\sqrt{2}} |0\rangle \xrightarrow{\text{CNOT}} \frac{|00\rangle + |11\rangle}{\sqrt{2}} = |\Phi^+\rangle$$

Example (parameterised ansatz):

$$|\psi(\theta)\rangle = R_y(\theta_3)R_y(\theta_2)\text{CNOT}R_y(\theta_1)R_y(\theta_0)|00\rangle$$

used to evaluate $\langle H \rangle = \langle \psi(\theta) | ZZ + 0.5 XI | \psi(\theta) \rangle$.

Step 2 — Optimize (Transpile)

Convert the abstract circuit to an **ISA (Instruction Set Architecture) circuit**:

1. **Gate decomposition** — rewrite abstract gates in hardware basis (e.g. `rz`, `sx`, `cx`)
2. **Routing** — insert SWAP gates to respect the hardware coupling map
3. **Optimization** — cancel redundant gates, reduce depth

Level	Description	Use case
0	No optimisation	Debug/testing
1	Light	Quick runs
2	Medium	General use
3	Heavy	Real hardware

Observables must also be remapped after transpilation:

```
H.isa = H.apply_layout(circuit.isa.layout)
```

Step 3 — Execute

Run ISA circuits using one of the two primitives from `qiskit-ibm-runtime`:

Primitive	Output	Use case
Sampler	Bitstring counts / probabilities	Grover's, QFT
Estimator	$\langle O \rangle$ expectation values	VQE, QAOA

Execution modes (real hardware):

- **Session** — reserved QPU connection for iterative algorithms (VQE, QAOA)
- **Batch** — parallel job submission for independent parameter sweeps

Step 4 — Post-process

Interpret raw primitive outputs to answer the original question:

- Plot histograms of bitstring distributions
- Apply readout-error mitigation techniques
- Marginalise distributions to smaller qubit subsets
- Postselect on physical constraints (parity, spin, particle number)
- In variational algorithms: feed $\langle H \rangle$ to a classical optimiser

Experiments

A — Bell State $|\Phi^+\rangle$ (Sampler)

Two-qubit entangled state. Measured 1024 shots via `BackendSamplerV2`. Expected & confirmed: only $|00\rangle$ and $|11\rangle$ appear ($\approx 50\%$ each); $|01\rangle$ and $|10\rangle$ are forbidden by entanglement.

$$P(|00\rangle) + P(|11\rangle) \approx 1.00$$

B — Pauli Observable (Estimator)

Parameterised 2-qubit ansatz evaluated over four θ configurations. Computed $\langle ZZ + 0.5 XI \rangle$ via `BackendEstimatorV2`. Energy landscape visualised — foundation of the Variational Quantum Eigensolver.

C — GHZ State (Full Pattern)

Three-qubit Greenberger–Horne–Zeilinger state:

$$|GHZ\rangle = \frac{1}{\sqrt{2}}(|000\rangle + |111\rangle)$$

Built with H on qubit 0 followed by a chain of CNOTs. Ran 2048 shots; only $|000\rangle$ and $|111\rangle$ observed, confirming maximal three-qubit entanglement.

Qiskit Tools Used

- `QuantumCircuit` — circuit construction
 - `SparsePauliOp` — Pauli observable definition
 - `ParameterVector` — parameterised ansatz circuits
 - `generate_preset_pass_manager` — transpilation (levels 0–3)
 - `BackendSamplerV2` — shot-based measurement counts
 - `BackendEstimatorV2` — Pauli expectation values
 - `AerSimulator` — local simulation (no IBM account needed)
 - `.apply_layout()` — remaps observable to physical qubit layout
-

Key Takeaways

1. **ISA Circuits are Mandatory.** Only circuits expressed in the hardware's native gate set and respecting its coupling map can run on IBM QPUs. The transpiler handles this automatically.
2. **Sampler vs Estimator.** Sampler returns bitstring counts (probability distributions); Estimator returns expectation values. Choose based on what the algorithm needs as output.
3. **Observable Layout Must Match.** After transpilation, logical-to-physical qubit remapping changes qubit indices. Always call `.apply_layout()` on your observable to keep it consistent.
4. **Higher Optimization = Fewer Gates.** Optimization level 3 can cut gate count significantly over level 0, reducing hardware noise — at the cost of longer classical compilation time.
5. **The Pattern is Modular.** Each of the four steps can be independently improved or swapped out, enabling seamless integration of new tools, error-mitigation techniques, and hardware backends.